

```

GNU nano 8.7.1                                     signal_handler.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_ARGS 32
#define BUF_SIZE 1024

volatile pid_t foreground_pid = 0;

// Trim leading/trailing whitespace
char *trim_space(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}

// Custom SIGINT (Ctrl+C) Signal Handler
void sigint_handler(int sig) {
    if (foreground_pid > 0) {
        // Forward SIGINT directly to the running foreground process
        kill(foreground_pid, SIGINT);
        printf("\n[Signal Handler] Forwarded SIGINT to foreground process (PID: %d)\n", foreground_pid);
    } else {
        // Protect shell: redisplay prompt without exiting
        printf("\n[Signal Handler] SIGINT caught. Shell protected. (No foreground job)\n");
        printf("SignalShell > ");
        fflush(stdout);
    }
}

```

```

// Register and configure signal handlers using sigaction
void setup_signal_handlers() {
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = sigint_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART; // Automatically restart interrupted system calls like fgets

    if (sigaction(SIGINT, &sa, NULL) < 0) {
        perror("sigaction registration failed");
        exit(EXIT_FAILURE);
    }
}

void execute_command(char *line) {
    line = trim_space(line);
    if (strlen(line) == 0) return;

    char *args[MAX_ARGS];
    int arg_count = 0;
    char *token = strtok(line, " \t");
    while (token != NULL && arg_count < MAX_ARGS - 1) {
        args[arg_count++] = token;
        token = strtok(NULL, " \t");
    }
    args[arg_count] = NULL;

    if (arg_count == 0) return;

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return;
    } else if (pid == 0) {
        // Child process resets SIGINT handler to default action (terminate)

```

```

        signal(SIGINT, SIG_DFL);

        execvp(args[0], args);
        perror("Command execution failed");
        exit(EXIT_FAILURE);
    } else {
        // Track active foreground child PID
        foreground_pid = pid;

        int status;
        waitpid(pid, &status, 0);

        if (WIFSIGNALED(status) && WTERMSIG(status) == SIGINT) {
            printf("\nForeground job (PID: %d) terminated by SIGINT.\n", pid);
        }

        // Reset foreground PID once job terminates
        foreground_pid = 0;
    }
}

int main() {
    char input_buf[BUF_SIZE];

    setup_signal_handlers();

    while (1) {
        printf("SignalShell > ");
        fflush(stdout);

        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline

        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;

        execute_command(input_buf);
    }

    return 0;
}

```

OUTPUT :

```
abhis@MIKHAIL-37:~$ nano signal_handler.c
abhis@MIKHAIL-37:~$ gcc -g -Wall signal_handler.c -o signal_handler
abhis@MIKHAIL-37:~$ ./signal_handler
SignalShell > ^C
[Signal Handler] SIGINT caught. Shell protected. (No foreground job)
SignalShell > sleep 30
^C
[Signal Handler] Forwarded SIGINT to foreground process (PID: 1125)
Foreground job (PID: 1125) terminated by SIGINT.
```

```
SignalShell > exit
abhis@MIKHAIL-37:~$ valgrind --leak-check=full ./signal_handler
==1140== Memcheck, a memory error detector
==1140== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1140== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1140== Command: ./signal_handler
==1140==
SignalShell > exit
==1140==
==1140== HEAP SUMMARY:
==1140==     in use at exit: 0 bytes in 0 blocks
==1140==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==1140==
==1140== All heap blocks were freed -- no leaks are possible
==1140==
==1140== For lists of detected and suppressed errors, rerun with: -s
==1140== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```